Virtual Prototyping and Sensor Integration of the Autobot Differential Drive Robot in ROS/Gazebo: A Detailed Implementation

Heytt Satra
Department of Computer Engineering
Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Mumbai, India
heyttsatra17@gmail.com
https://orcid.org/0009-0008-2443-9110

Daksh Rungta
Department of Computer Engineering
Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Mumbai, India
daksh.rungta07@nmims.in
https://orcid.org/0009-0000-0161-1338

Shubham Tanna
Department of Computer Engineering
Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Mumbai, India
shubhamjtanna@gmail.com
https://orcid.org/0009-0009-4448-0329

Aadit Botadra
Department of Computer Engineering
Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Mumbai, India
aaditbotadra@gmail.com
https://orcid.org/0009-0003-4807-1270

Sunny Nanade*
Department of Mechatronics Engineering
Mukesh Patel School of Technology Management & Engineering, SVKM's NMIMS, Mumbai, India
sunny.nanade@nmims.edu
https://orcid.org/0000-0001-7098-1084

***Abstract*—Robotic simulations are pivotal in developing and testing autonomous systems, offering cost-effective platforms for algorithm development and sensor integration. This paper presents a detailed account of implementing a differential drive robot (DDR) simulation, named *autobot*, within the Robot Operating System (ROS) and Gazebo environment. A key focus is on virtual prototyping, allowing for rapid design iteration. The simulation emphasizes sensor integration, incorporating a 2D LIDAR, a standard camera, and a depth camera. The *autobot*'s physical attributes are meticulously defined using a detailed Unified Robot Description Format (URDF) model. This project provides a robust and reusable open-source platform for exploring sensor data processing, algorithm development, and the virtual validation of robotic systems. The paper thoroughly discusses the nuances of model implementation, sensor configuration, and potential avenues for future simulation fidelity enhancements.**

***Index Terms*—Robotics Simulation, Differential Drive Robot (DDR), ROS/Gazebo, Sensor Integration, Virtual Prototyping, URDF**

* Corresponding author.

I. Introduction

The field of robotics has witnessed remarkable advancements, spurred by the increasing capabilities of autonomous systems. A cornerstone of this progress is the effective utilization of robotic simulations across a diverse range of applications, including autonomous navigation, Simultaneous Localization and Mapping (SLAM), path planning, control systems development, and robotics education. Simulation environments offer a compelling constellation of advantages that have fun-

damentally reshaped the landscape of robotics research and development. Chief among these are cost-effectiveness, safety, and accelerated development cycles. By enabling the virtual instantiation of robotic systems, simulations obviate the need for expensive and time-consuming hardware prototypes in the early stages of a project. This allows researchers and engineers to explore a wide range of design possibilities and algorithmic strategies without incurring significant financial burdens or risking damage to physical equipment. Furthermore, simulations provide a safe haven for exploring hazardous or unpredictable scenarios, enabling the rigorous testing of robot behavior in situations that would be impractical or dangerous to replicate in the real world. The ability to iterate rapidly in a virtual environment drastically shortens development cycles, allowing for quicker refinement and optimization of algorithms before their deployment on physical robots.

Among the myriad robotic platforms available, differential drive robots (DDRs) have consistently emerged as a particularly attractive and widely adopted choice. Their popularity stems from their inherent simplicity in design and exceptional maneuverability, making them well-suited for a variety of tasks in diverse environments. A DDR achieves motion by independently controlling the rotational velocities of two coaxially aligned wheels, enabling precise control over its linear and angular movements. This relatively simple mechanical configuration translates to a platform that is both straightforward to construct, control, and maintain, thus facilitating its widespread adoption across a broad spectrum of applications. These applications range from warehouse automation and automated guided vehicles (AGVs) in industrial settings to delivery services, assistive robotics, educational platforms, and research prototypes. Moreover, the inherent agility of DDRs allows them to navigate confined spaces and execute intricate maneuvers with relative ease, further solidifying their relevance in dynamic and complex environments.

The *autobot* project was conceived with the primary objective of creating a meticulously detailed and easily accessible simulation environment within which to explore and validate various core concepts and emerging trends in mobile robotics. Recognizing that the physical instantiation of a robot inevitably presents a multitude of complex engineering challenges, we sought to leverage the power of the virtual domain. The virtual domain offers a readily modifiable, fully controllable, and inherently safe environment that is ideally suited for iterative design, algorithmic validation, and comprehensive system testing. Specifically, this project aimed to develop a realistic and versatile simulation environment for the *autobot* robot, facilitating the testing and refinement of fundamental autonomous navigation algorithms. More generally, this project can serve as a valuable educational resource for students and researchers alike. The underlying motivation stemmed from the desire to provide an open-source, well-documented, and easily adaptable example of a DDR simulation that can be readily tailored to a diverse range of research and development purposes.

The contributions of this paper are multifaceted. First, it offers a detailed account of the implementation of a complete *autobot* robot model within the Gazebo simulation environment. This includes a precise definition of its physical parameters (dimensions, mass distribution, etc.) and a representative sensor configuration. The simulated *autobot* is equipped with a suite of sensors, including a 2D LIDAR, a standard camera, and a depth camera. All are capable of providing rich sensory data that can be used for various algorithm development activities. The robot model is carefully constructed based on assumed physical characteristics and dimensions, reflecting common DDR designs. Second, the paper presents a comprehensive discussion of the challenges encountered and the invaluable lessons learned throughout the simulation process, providing actionable insights into the intricacies of creating realistic and effective robotic simulations. This includes a detailed description of how a keyboard teleoperation node is configured in conjunction with the Rviz visualization tool. The combined tool is set up to read and simulate the environment captured by these sensors and test a wide array of sensors in real-time. This teleoperation integration allows researchers to manually control the robot and observe sensor readings, facilitating intuitive debugging and evaluation of sensor performance. Finally, the paper concludes by outlining promising avenues for future work and potential enhancements to the *autobot* simulation environment, providing a roadmap for continued development and exploration.

The remainder of this paper is organized as follows. Section II provides the reader with the necessary background information, setting the stage by briefly reviewing related simulation projects that leverage the Robot Operating System (ROS) and the Gazebo simulator. Section III then elucidates the details of the physical model of the *autobot* robot, and focuses on the intricacies of the software environment setup, describing the configuration of various sensors and the integration of teleoperation capabilities. Finally, Section IV concludes the paper by summarizing the key contributions, highlighting the lessons learned, and outlining promising avenues for future work and potential enhancements to the *autobot* simulation environment.

II. Physical Model of *Autobot* and Simulation Environment Setup

A. Robot Overview

The *autobot*, depicted in Figure 1, is a simulated differential drive robot designed primarily for algorithm testing within the Gazebo environment. The robot features a rectangular chassis, exhibiting approximate dimensions of 25 cm in length and 20 cm in width. This design allows for a compact footprint, and is designed to carry a wide variety of sensor devices. Two independently driven wheels are mounted on either side of the chassis, providing the means for motion control. The simulation serves as the base for exploration of all types of autonomous technologies. The kinematic model of this DDR has been discussed in earlier sections of this paper. This paper seeks to highlight the other features and applications that can be tested in the environment. The key component of this

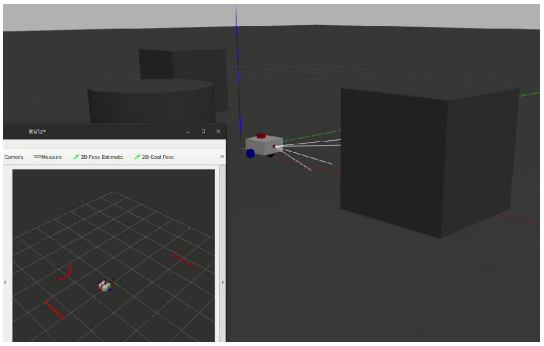


Fig. 1. Gazebo rendering of the *autobot* model, highlighting key components: chassis and wheels.

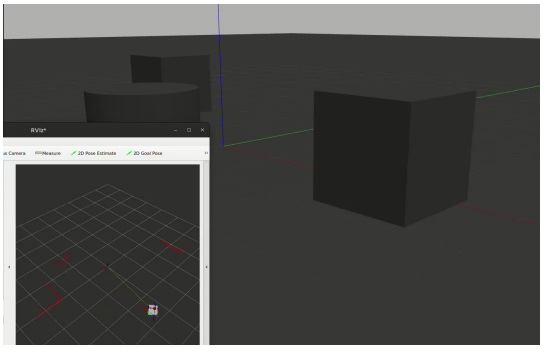


Fig. 2. Wide-angle shot of the Gazebo environment showcasing the *autobot* amongst the objects it interacts with.

feature is using simulations to test the sensors in the virtual domain.

B. Simulation Environment Setup

1) Software and Hardware: The *autobot* simulation environment was developed using the Robot Operating System (ROS) Noetic Ninjemys, the Gazebo 11 simulator, and the Ubuntu 20.04 operating system. These tools provide a robust and widely used platform for robotics simulation and development, ensuring reproducibility and compatibility with existing ROS-based software. The choice of ROS Noetic is motivated by its stability, long-term support, and extensive community resources, making it an ideal platform for both research and education. Gazebo 11, a powerful 3D robotics simulator, provides a realistic environment for simulating the robot's physical interactions with the world, including its dynamics, sensor behavior, and environmental conditions. The use of Ubuntu 20.04 ensures compatibility and consistency across different development environments.

2) Gazebo Model Creation: The *autobot* robot model within Gazebo is defined using the Unified Robot Description Format (URDF). The URDF file specifies the robot's physical structure, including its links, joints, and inertial properties. This description allows Gazebo to accurately simulate the robot's behavior. The model is then configured by a controller plugin to ensure the robot can receive movement and operation commands from a ROS node. The URDF encompasses crucial information, such as the robot's dimensions, mass distribution, joint types, and sensor attachments. The level of detail incorporated into the URDF directly impacts the fidelity of the simulation, allowing researchers to fine-tune the model to accurately reflect the physical characteristics of a real-world robot.

Figure 2 displays the Gazebo environment, showcasing the *autobot* in a scene with various obstacles. In this environment, the robot utilizes the following sensors to visualise its internal and external world: a 2D LIDAR, a Camera and a Depth Camera. To operate it better, we add a keyboard node, where a keyboard is used as teleop controller. This has greatly helped improve our system, and we are able to easily obtain information from all types of sensors.

3) Sensor Integration: The *autobot* simulation integrates a 2D LIDAR, a Camera, and a Depth Camera, each providing distinct sensory data:

LIDAR Integration:

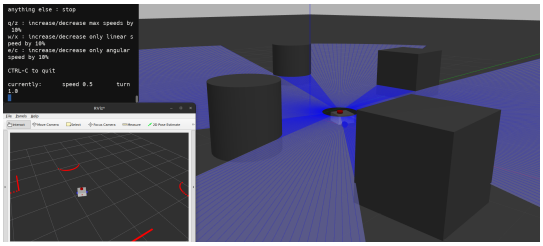


Fig. 3. Screenshot of simulated LIDAR data visualized in Rviz. The LIDAR data shows how the robot perceives the presence of other objects in its surrounding area.

The *autobot* is equipped with a simulated 2D LIDAR sensor and visualized through the Rviz Tool, as displayed through Figure 3. This configuration allows for the perception of the surrounding environment, and the use of the LIDAR data can be used in future configurations of this simulation to enable such features as obstacle detection, mapping, and localization. The LIDAR sensor is modeled using the `gazebo_ros_ray_sensor` plugin within Gazebo. This plugin simulates the behavior of a laser rangefinder by casting virtual laser beams and measuring the distance to the first obstacle encountered.

Camera Integration:

Figures 4 highlights the images a camera would provide in a simulated environment. Cameras are an essential component in creating autonomous technologies, since the image provided by a camera can be fed into neural networks and perform functions from computer vision to image analysis. This sensor is especially helpful in understanding concepts such as recognition, and understanding of objects. By adding the teleoperation keyboard, one is able to make sense of what is being captured by the device. The use of the teleoperation keyboard, creates a way for one to intuitively check their sensors, by having the robot manually look around, and having the ability to view it.

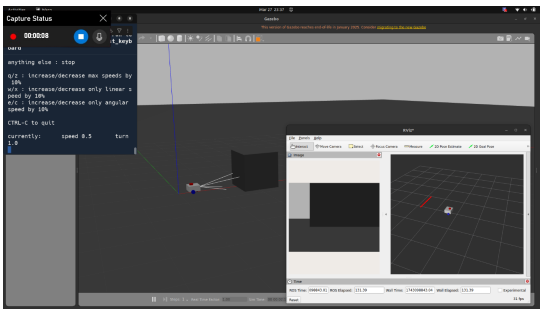


Fig. 4. Teleoperation of the Camera in both Rviz and Gazebo, visualizing its view of the objects in its path.

Figure 4 showcases the image that the camera is creating from the Rviz tool and in the Gazebo Simulation. To control the robot, we used a keyboard node, helping visualize what the robot was viewing through its camera. The red color markings, indicates the direction that the robot is going in, and helps assist the user understand what is being visualised

Depth Camera Integration:

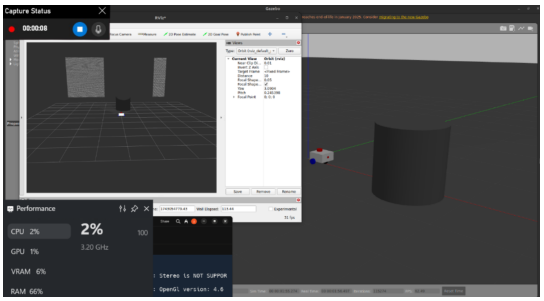


Fig. 5. Teleoperation of the Depth Camera, visualising a combined Rviz and Gazebo view of the objects in its path.

Figure 5 showcases the depth camera in the Rviz and Gazebo environments through teleoperation. What is being captured here, is the depth and range of the objects that it is looking at, in addition to the normal data. The Gazebo shot shows the surrounding environment, whilst the shot in Rviz captures the map that the camera is creating. This helps the robot scan and read information from all types of environments. It can also determine which distance each object is at from the robot itself.

4) Teleoperation Nodes: The keyboard teleoperation (teleop) node allows direct control of the *autobot*'s wheel velocities. Subscribing to `geometry_msgs/Twist` messages, which contain the desired linear and angular velocities for the robot, a keyboard can be used as a tool to move the robot, as explained above. Using the Rviz and Gazebo tools, this was accomplished and was visualized through both, the various sensors and the simulations themselves. The teleop node facilitates rapid prototyping of navigation, and also is helpful in the visualisation and configuration of data, as used in the above situations. The integration of teleoperation has been one of the most important steps of creating a successful simulation of the physical robot.

III. Conclusion

This paper detailed the development of the *autobot*, a simulated differential drive robot within the ROS/Gazebo environment. The simulation incorporates a URDF model and simulated sensor data from a 2D LIDAR, a camera, and a depth camera. This simulation provides the starting point for various autonomous technologies to be tested. In essence, this paper describes how different types of sensors, and more can be added to this, so that a comprehensive dataset can be obtained and used for different activities. The use of ROS and Gazebo ensures the reproducibility of the project and the scalability for users to build on top of it. The paper also visualises and simulates what a developer can expect while creating these models. Lastly, with the addition of the keyboard as a controller, the ability for someone to easily test new robots without having to code is possible. This method of integration, is a good start for new beginners to start creating more autonomous robots!

Acknowledgment

The authors gratefully acknowledge the Articulated Robotics channel on YouTube for providing accessible and easily understandable tutorials that aided in the foundational understanding of mobile robot design principles explored within this research.

References

[1] The Github repo for this project: https://github.com/heytt-satra/my_bot

[2] This Github repo was used as inspiration for the project: https://github.com/Tanmetry554/articubot_one